Inheritance

Pengertian

Inheritance merupakan mekanisme di mana sebuah kelas baru (*subclass*) mewarisi atribut dan method dari kelas yang sudah ada (*superclass*). Subclass tidak perlu menulis ulang apa yang sudah dimiliki superclass, dan hanya menambahkan atau mengubah hal yang membedakannya.

Pada materi UML class relationships kita sudah mengenal tiga hubungan antar kelas: asosiasi, agregasi, dan komposisi. Inheritance adalah hubungan **keempat**, dan ada satu perbedaan penting:

- Asosiasi, agregasi, dan komposisi menghubungkan objek dengan objek (Nasabah *menggunakan* MesinATM, Bank *memiliki* Karyawan).
- Inheritance menghubungkan kelas dengan kelas (RekeningTabungan *adalah jenis dari* Rekening).

Jenis	Kata Kunci	Kekuatan	Contoh Nyata
Asosiasi	“Menggunakan”	Lemah	Nasabah menggunakan mesin ATM
Agregasi	“Memiliki”	Sedang	Bank memiliki Karyawan
Komposisi	“Terdiri dari”	Kuat	Rekening terdiri dari Catatan Transaksi
Pewarisan	“Adalah jenis dari”	Sangat Kuat (antar kelas)	Rekening Tabungan adalah jenis Rekening

Dalam konsep inheritance, terdapat 2 pihak utama:

1. Superclass / Parent Class

   Kelas yang menurunkan atribut dan method-nya.

2. Subclass / Child Class

   Kelas hasil turunan dari kelas lain yang secara otomatis mendapatkan atribut serta method dari parent class-nya.

Jika relasi *is-a* tidak bisa diterapkan secara logis, maka inheritance sebaiknya tidak digunakan gunakan komposisi sebagai alternatif.

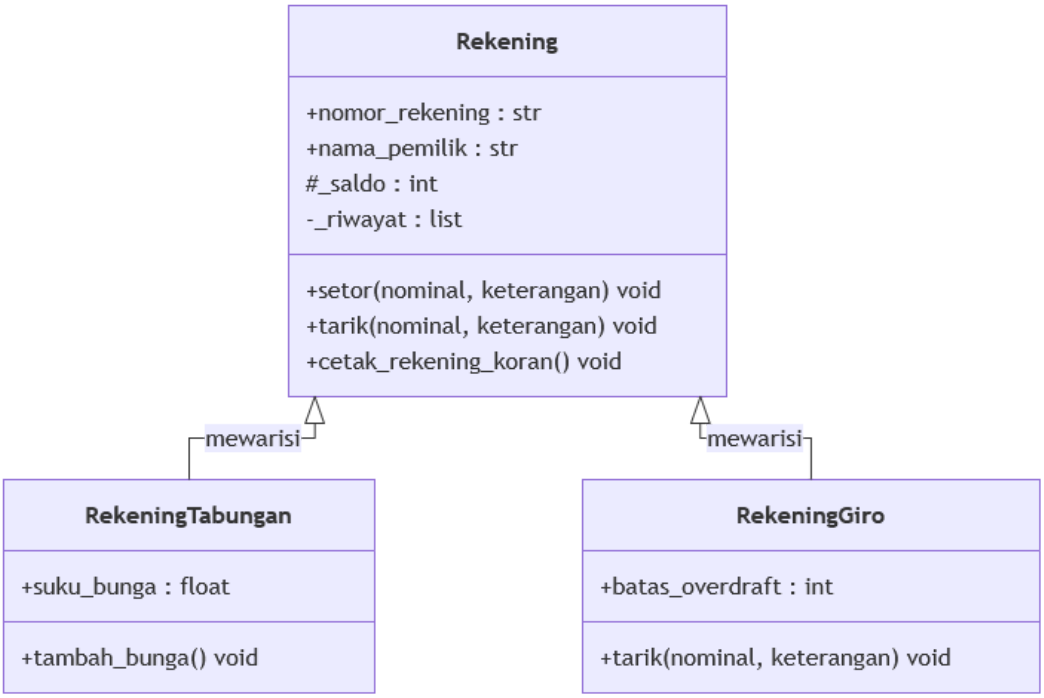
Inheritance diterapkan saat beberapa class memiliki basis (atribut/method) yang sama, namun juga punya perbedaan spesifik masing-masing. Contoh: Sebuah bank memiliki berbagai jenis rekening, misalnya Tabungan dan Giro. Keduanya sama-sama punya nomor rekening, nama pemilik, saldo, serta kemampuan setor dan tarik. Perbedaannya hanya pada hal spesifik: Tabungan punya bunga, Giro punya batas overdraft. Tanpa inheritance, atribut dan method yang sama harus ditulis ulang di setiap kelas.

Pengujian “Is-a”

Jadi, sebelum kita menggunakan *inheritance* kita harus dapat memahami terlebih dahulu apakah relasi *is-a* masuk akal, kemudian kita bandingkan dengan ketiga relasi sebelumnya.

Pernyataan	Hasil	Relasi yang tepat
Teller adalah Karyawan	Benar	*Inheritance* (Pewarisan)
Rekening Giro adalah Rekening	Benar	*Inheritance* (Pewarisan)
Bank adalah Karyawan	Salah	Bank memiliki Karyawan (Agregasi)
Rekening adalah Nasabah	Salah	Nasabah memiliki akun Rekening (Asosisasi)
Catatan Transaksi adalah Rekening	Salah	Rekening terdiri dari Catatan Transaksi (Komposisi)

Gambaran Relasi



Notasi: garis solid dengan segitiga kosong di sisi *SuperClass*, Panah selalu menunjuk ke induk atau *Parent Class*.

Ciri-ciri Pewarisan

1. *Subclass* otomatis memiliki atribut dan method milik *superclass*.
2. *Subclass* dapat menambahkan atribut dan method baru miliknya sendiri.
3. *Subclass* dapat menimpa (*Override*) method *superclass* dengan perilaku yang berbeda.
4. *Subclass* memanggil konstruktor *superclass* melalui “super().__init__()”.
5. Hubungannya antar kelas, satu objek RekeningGiro sekaligus adalah objek Rekening, bukan dua objek yang terpisah.

Implementasi

Superclass: Rekening

Catatan Transaksi sama persis dengan materi UML sebelumnya. Rekening juga masih memakai komposisi _riwayat, hanya saldonya dibuat protected (_saldo) supaya bisa diakses subclass.

```
class CatatanTransaksi:
    """Sama seperti materi UML class relationships."""

    def __init__(self, id_transaksi, tipe, nominal, keterangan):
        self.id_transaksi = id_transaksi
        self.tipe = tipe
        self.nominal = nominal
        self.keterangan = keterangan

    def __str__(self):
        simbol = "+" if self.tipe == "KREDIT" else "-"
        return f"[{self.id_transaksi}] {self.tipe:<6}
{simbol}Rp{self.nominal:,} | Ket: {self.keterangan}"


class Rekening:
    """Superclass: atribut dan method yang dimiliki SEMUA jenis
rekening."""

    def __init__(self, nomor_rekening, nama_pemilik, saldo_awal=0):
        self.nomor_rekening = nomor_rekening
        self.nama_pemilik = nama_pemilik
        self._saldo = saldo_awal      # protected: boleh diakses subclass
        self._riwayat = []            # komposisi dengan CatatanTransaksi

        if saldo_awal > 0:
            self._buat_catatan("KREDIT", saldo_awal, "Setoran awal
pembukaan rekening")

    @property
    def saldo(self):
```

```
        return self._saldo

    def _buat_catatan(self, tipe, nominal, keterangan):
        id_baru = f"TRX-{len(self._riwayat) + 1:04d}"
        self._riwayat.append(CatatanTransaksi(id_baru, tipe, nominal,
keterangan))

    def setor(self, nominal, keterangan="Setor tunai"):
        if nominal <= 0:
            print("  Nominal setoran tidak valid.")
            return
        self._saldo += nominal
        self._buat_catatan("KREDIT", nominal, keterangan)
        print(f"  Setoran Rp{nominal:,} berhasil dicatat.")

    def tarik(self, nominal, keterangan="Tarik tunai"):
        if nominal <= 0:
            print("  Nominal penarikan tidak valid.")
        elif nominal > self._saldo:
            print("  Saldo tidak mencukupi untuk transaksi.")
        else:
            self._saldo -= nominal
            self._buat_catatan("DEBET", nominal, keterangan)
            print(f"  Penarikan Rp{nominal:,} berhasil dicatat.")

    def cetak_rekening_koran(self):
        print(f"\n  Rekening Koran: {self.nomor_rekening} an
{self.nama_pemilik}")
        print(f"  Jenis Rekening: {type(self).__name__}")
        print(f"  Saldo Akhir: Rp{self._saldo:,}")
        print("  Riwayat Mutasi Transaksi:")
        for trx in self._riwayat:
            print(f"    {trx}")
```

Superclass: RekeningTabungan dan RekeningGiro

```
class RekeningTabungan(Rekening):
    """Subclass: menambah suku bunga dan method tambah_bunga()."""
```

```
    def __init__(self, nomor_rekening, nama_pemilik, saldo_awal=0,
suku_bunga=0.02):
        super().__init__(nomor_rekening, nama_pemilik, saldo_awal)  #
panggil konstruktor superclass
        self.suku_bunga = suku_bunga

    def tambah_bunga(self):
        bunga = int(self._saldo * self.suku_bunga)
        self._saldo += bunga
        self._buat_catatan("KREDIT", bunga, "Bunga tabungan")
        print(f"  Bunga Rp{bunga:,} ditambahkan.")


class RekeningGiro(Rekening):
    """Subclass: menambah batas overdraft dan meng-override method
tarik()."""

    def __init__(self, nomor_rekening, nama_pemilik, saldo_awal=0,
batas_overdraft=1000000):
        super().__init__(nomor_rekening, nama_pemilik, saldo_awal)
        self.batas_overdraft = batas_overdraft

    def tarik(self, nominal, keterangan="Tarik tunai"):   # method
overriding
        if nominal <= 0:
            print("  Nominal penarikan tidak valid.")
        elif nominal > self._saldo + self.batas_overdraft:
            print("  Melebihi batas overdraft.")
        else:
            self._saldo -= nominal
            self._buat_catatan("DEBET", nominal, keterangan)
            print(f"  Penarikan Rp{nominal:,} berhasil dicatat.")
```

Penggunaan

```
tabungan = RekeningTabungan("554-900-112", "Rina Marlina", 2000000, 0.02)
tabungan.setor(500000, "Transfer masuk dari klien")   # method milik
```

```
Rekening (diwarisi)
tabungan.tambah_bunga()                                    # method milik
RekeningTabungan
tabungan.cetak_rekening_koran()

giro = RekeningGiro("880-100-200", "Budi Santoso", 500000, 1000000)
giro.tarik(1200000, "Pembayaran supplier")                 # versi override:
boleh melewati saldo
giro.cetak_rekening_koran()

# Cek relasi pewarisan
print(isinstance(tabungan, Rekening))                # True  -> Tabungan adalah
Rekening
print(isinstance(tabungan, RekeningGiro))            # False
print(issubclass(RekeningGiro, Rekening))            # True
```

super() dan Method Overriding

super() dipakai subclass untuk memanggil konstruktor atau method milik superclass. Pada RekeningTabungan, super().__init__(...) mengisi atribut nomor_rekening, nama_pemilik, dan _saldo lewat konstruktor Rekening, sehingga subclass hanya perlu mengisi atribut miliknya (suku_bunga).

Method overriding terjadi ketika subclass mendefinisikan ulang method dengan nama yang sama seperti superclass. Contohnya tarik() pada RekeningGiro, yang aturannya berbeda dari tarik() milik Rekening. Python akan memakai versi milik subclass.

Tipe-Tipe Inheritance

1. Single Inheritance

Satu subclass, satu superclass. Contoh: RekeningTabungan(Rekening) diatas.

2. Multilevel Inheritance

Pewarisan berjenjang: Rekening -> RekeningTabungan -> TabunganBerjangka.

```
class TabunganBerjangka(RekeningTabungan):
    """Tabungan yang dikenai denda jika ditarik sebelum jatuh tempo."""

    def __init__(self, nomor_rekening, nama_pemilik, saldo_awal=0,
suku_bunga=0.05, jangka_bulan=12):
        super().__init__(nomor_rekening, nama_pemilik, saldo_awal,
suku_bunga)
        self.jangka_bulan = jangka_bulan

    def tarik(self, nominal, keterangan="Tarik tunai"):
        print("  Penarikan sebelum jatuh tempo dikenakan denda Rp50,000.")
        super().tarik(nominal + 50000, keterangan + " (termasuk denda)")


deposito = TabunganBerjangka("554-777-001", "Rina Marlina", 10000000, 0.05,
12)
deposito.tarik(1000000)      # memanggil versi TabunganBerjangka, lalu
Rekening lewat super()
deposito.tambah_bunga()      # diwarisi dari RekeningTabungan
deposito.setor(500000)       # diwarisi dari Rekening
```

3. Hierarchical Inheritance

Satu superclass diwarisi oleh beberapa subclass: Rekening diwarisi oleh RekeningTabungan dan RekeningGiro (lihat diagram UML di atas).

4. Multilevel Inheritance

Satu subclass mewarisi dari lebih dari satu superclass. Contoh: rekening tabungan yang juga punya layanan mobile banking.

```
class LayananMobileBanking:
    """Kemampuan tambahan (mixin), bukan jenis rekening."""

    def transfer_digital(self, rekening_tujuan, nominal):
        print(f"  [Mobile] Transfer ke
```

```
{rekening_tujuan.nomor_rekening}...")
        if nominal > self.saldo:
            print("  Transfer gagal: saldo tidak mencukupi.")
            return
        self.tarik(nominal, f"Transfer ke
{rekening_tujuan.nomor_rekening}")
        rekening_tujuan.setor(nominal, f"Transfer dari
{self.nomor_rekening}")

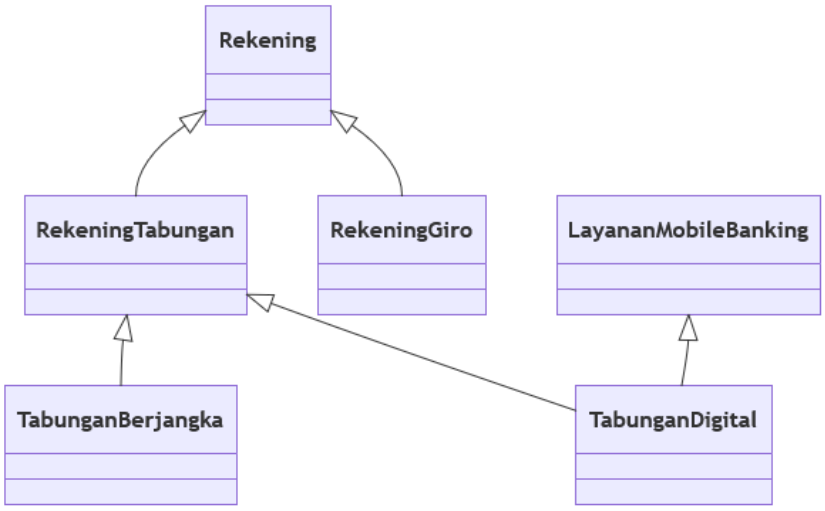

class TabunganDigital(RekeningTabungan, LayananMobileBanking):
    pass


digital = TabunganDigital("554-500-999", "Siti Rahma", 3000000)
digital.transfer_digital(tabungan, 250000)

print([k.__name__ for k in TabunganDigital.mro()])
# ['TabunganDigital', 'RekeningTabungan', 'Rekening',
'LayananMobileBanking', 'object']
```

Urutan kelas di dalam tanda kurung menentukan **MRO (Method Resolution Order)**, yaitu urutan pencarian method oleh Python.

UML untuk keempat tipe:



Inheritance dan Encapsulation

Pada Rekening, saldo ditulis _saldo (protected), bukan __saldo (private). Alasannya, atribut private mengalami name mangling sehingga subclass juga tidak bisa mengaksesnya.

```
class Rekening:
    def __init__(self, nomor_rekening, nama_pemilik, saldo_awal=0):
        self.__saldo = saldo_awal          # private


class RekeningGiro(Rekening):
    def tarik(self, nominal):
        self.__saldo -= nominal            # AttributeError:
        # 'RekeningGiro' object has no attribute '_RekeningGiro__saldo'
```

Jadi atribut yang perlu dipakai subclass sebaiknya *protected* (_), sedangkan atribut yang benar-benar rahasia milik superclass dibuat *private* (__).

Menggabungkan Semua Relasi dalam Satu Sistem Bank

Sistem perbankan yang utuh memakai keempat relasi sekaligus. Karyawan dan Bank sama seperti materi UML sebelumnya, sedangkan Teller dan ManajerCabang adalah subclass baru dari Karyawan.

```
class Teller(Karyawan):
    def __init__(self, nama, nip):
        super().__init__(nama, nip, "Teller")

    def layani_setoran(self, rekening, nominal):
        # Asosiasi: rekening diterima sebagai parameter method
        print(f"  Teller {self.nama} melayani setoran...")
        rekening.setor(nominal, f"Setoran via teller {self.nip}")
```

```
class ManajerCabang(Karyawan):
    def __init__(self, nama, nip):
        super().__init__(nama, nip, "Branch Manager")

    def setujui_overdraft(self, rekening_giro, batas_baru):
        rekening_giro.batas_overdraft = batas_baru
        print(f"  {self.nama} menyetujui batas overdraft Rp{batas_baru:,}
untuk {rekening_giro.nama_pemilik}")


class MesinATM:
    def __init__(self, id_atm, lokasi, saldo_kas):
        self.id_atm = id_atm
        self.lokasi = lokasi
        self.saldo_kas = saldo_kas

    def proses_penarikan(self, rekening, jumlah):
        if jumlah > self.saldo_kas:
            print(f"  [ATM {self.id_atm}] Saldo kas mesin tidak
mencukupi.")
            return
        saldo_sebelum = rekening.saldo
        rekening.tarik(jumlah, f"Tarik tunai ATM {self.id_atm}")
        if rekening.saldo < saldo_sebelum:
            self.saldo_kas -= jumlah


class Nasabah:
    def __init__(self, nama):
        self.nama = nama
        self._daftar_rekening = []       # Asosiasi: rekening dibuat di
luar, lalu didaftarkan

    def tambah_rekening(self, rekening):
        self._daftar_rekening.append(rekening)

    def tarik_tunai(self, atm, rekening, jumlah):
        # Asosiasi: MesinATM dan Rekening diterima sebagai parameter
        print(f"  {self.nama} memasukkan kartu ke ATM {atm.id_atm}
```

```
({atm.lokasi})...")
        atm.proses_penarikan(rekening, jumlah)
```

Penggunaan

```
# Agregasi + Pewarisan: Bank menampung Teller dan ManajerCabang (keduanya
adalah Karyawan)
bank = Bank("Bank Nasional Mandiri", "BNM")
teller = Teller("Andi Wijaya", "EMP01")
manajer = ManajerCabang("Dewi Lestari", "EMP02")
bank.tambah_karyawan(teller)
bank.tambah_karyawan(manajer)
bank.tampilkan_daftar_karyawan()

# Pewarisan + Komposisi: dua jenis rekening, masing-masing punya
CatatanTransaksi sendiri
tabungan = RekeningTabungan("554-900-112", "Rina Marlina", 2000000, 0.02)
giro = RekeningGiro("880-100-200", "Budi Santoso", 500000)

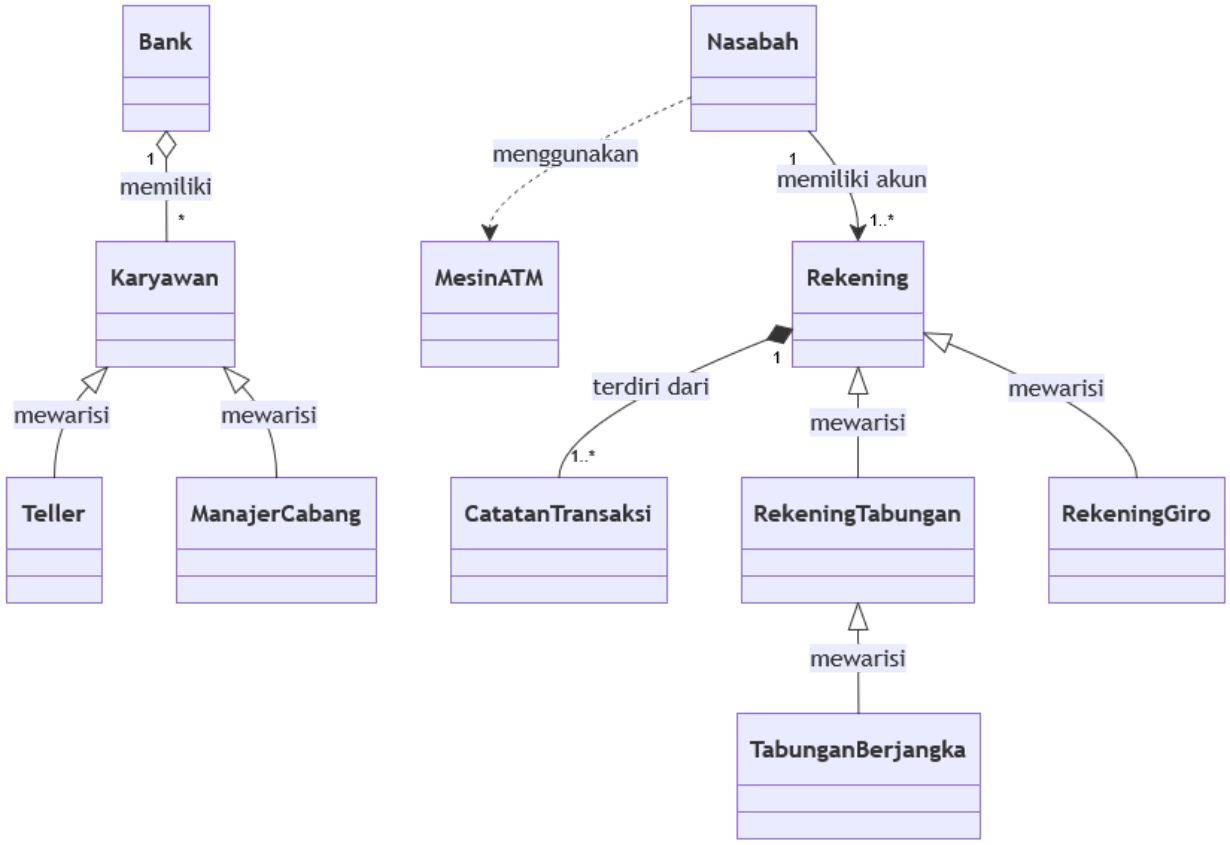
# Asosiasi: nasabah mendaftarkan rekening miliknya
rina = Nasabah("Rina Marlina")
rina.tambah_rekening(tabungan)
budi = Nasabah("Budi Santoso")
budi.tambah_rekening(giro)

# Asosiasi: teller melayani setoran, manajer menyetujui overdraft
teller.layani_setoran(tabungan, 500000)
manajer.setujui_overdraft(giro, 1500000)

# Asosiasi: nasabah menggunakan ATM. Method tarik() dipanggil di kedua
rekening,
# tetapi perilakunya berbeda karena RekeningGiro meng-override tarik()
atm = MesinATM("ATM-01", "Kantor Cabang Sudirman", 50000000)
rina.tarik_tunai(atm, tabungan, 300000)
budi.tarik_tunai(atm, giro, 1800000)

tabungan.cetak_rekening_koran()
giro.cetak_rekening_koran()
```

Diagram UML



Cara Membaca

Bank memiliki banyak Karyawan (agregasi), dan Teller serta ManajerCabang adalah Karyawan (pewarisan). Nasabah menggunakan MesinATM dan memiliki akun Rekening (asosiasi). Rekening terdiri dari CatatanTransaksi (komposisi), dan RekeningTabungan serta RekeningGiro adalah Rekening (pewarisan).

Perbandingan Lengkap

Aspek	Asosiasi	Agregasi	Komposisi	Pewarisan
Kata kunci	“Menggunakan”	“Memiliki”	“Terdiri dari”	“Adalah jenis dari”
Notasi UML	. . >	O - -	* - -	< | - -
Yang dihubungkan	Objek dengan Objek	Objek dengan Objek	Objek dengan Objek	Kelas dengan Kelas
Penulisan di Python	Objek diterima lewat parameter method	Objek dikirim ke penampung (list)	Objek dibuat di dalam Objek Induk	Class sub(Super):
Siklus Hidup	Masing-masing sendiri	Bagian tetap ada jika Induk dihapus	Bagian ikut musnah bersama Induk	Satu objek utuh (tidak ada dua objek terpisah)
Contoh	Nasabah dan mesin ATM	Bank dan Karyawan	Rekening dan Catatan Transaksi	RekeningGiro dan Rekening